3.2 Data Binding in Angular

Introduction

Data binding is one of the most powerful features in Angular. It establishes a connection between the component's data (TypeScript class) and the view (HTML template), enabling automatic synchronization. Angular provides several types of data binding, each serving specific purposes in creating dynamic, interactive applications.

Types of Data Binding

Angular supports four primary types of data binding:

- **String Interpolation** - One-way (Component → View)
- **Property Binding** - One-way (Component → View)
- **Event Binding** - One-way (View → Component)
- **Two-way Binding** - Bidirectional (Component ↔ View)

String Interpolation

String interpolation uses double curly braces `{{ }}` to display component data in the template. It is the simplest form of one-way data binding from the component to the view.

Basic String Interpolation

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-welcome',
  template: `
    <div>
      <h3>Welcome Message</h3>

      <!-- Simple property interpolation -->
      <p>Welcome, {{ userName }}!</p>
```

```
      <!-- Expression interpolation -->
      <p>Your account status: {{ isActive ? 'Active' : 'Inactive' }}</p>

      <!-- Method call interpolation -->
      <p>Current time: {{ getCurrentTime() }}</p>

      <!-- Arithmetic expression -->
      <p>Total score: {{ score1 + score2 + score3 }}</p>

      <!-- Object property interpolation -->
      <p>Email: {{ userProfile.email }}</p>
    </div>
  `
})
export class WelcomeComponent {
  userName: string = 'Jennifer Martinez';
  isActive: boolean = true;
  score1: number = 85;
  score2: number = 92;
  score3: number = 78;
  userProfile = {
    email: 'jennifer.martinez@example.com',
    level: 'Premium'
  };

  getCurrentTime(): string {
    return new Date().toLocaleTimeString();
  }
}
```

Output:

Browser Display:

Welcome Message

Welcome, Jennifer Martinez!

Your account status: Active

Current time: 2:45:30 PM

> Total score: 255
>
> Email: jennifer.martinez@example.com
>
> **Data Flow:** Component → View (One-way)

Property Binding

Property binding uses square brackets `[property]="value"` to set element properties dynamically. It binds component properties to DOM element properties, directive properties, or component properties.

Element Property Binding

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-property-demo',
  template: `
    <div>
      <h3>Property Binding Examples</h3>

      <!-- Image source binding -->
      <img [src]="imageUrl" [alt]="imageAlt" [width]="imageWidth">

      <!-- Button disabled state -->
      <button [disabled]="isProcessing">
        {{ isProcessing ? 'Processing...' : 'Submit' }}
      </button>

      <!-- Input value binding -->
      <input type="text" [value]="defaultText" [readonly]="isReadOnly">

      <!-- Link href binding -->
      <a [href]="websiteUrl" [target]="linkTarget">Visit Website</a>

      <!-- Textarea placeholder binding -->
      <textarea [placeholder]="placeholderText" [rows]="rowCount"></textarea>
    </div>
  `
})
export class PropertyDemoComponent {
```

```
  imageUrl: string = 'https://angular.io/assets/images/logos/angular/angular.png';
  imageAlt: string = 'Angular Logo';
  imageWidth: number = 150;

  isProcessing: boolean = false;

  defaultText: string = 'Default value';
  isReadOnly: boolean = true;

  websiteUrl: string = 'https://angular.io';
  linkTarget: string = '_blank';

  placeholderText: string = 'Enter your comments here...';
  rowCount: number = 5;
}
```

Output:

Browser Display:

Property Binding Examples

Submit

Default value

Visit Website

Key Points:

- Properties are enclosed in square brackets
- Values come from component class
- Updates automatically when component data changes

Class and Style Binding

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-style-binding',
  template: `
    <div>
      <h3>Class and Style Binding</h3>

      <!-- Single class binding -->
      <p [class.highlight]="isHighlighted">This text can be highlighted</p>

      <!-- Multiple class binding -->
      <div [class]="currentClasses">Multiple classes applied</div>

      <!-- Style binding -->
      <p [style.color]="textColor">Colored text</p>
      <p [style.font-size.px]="fontSize">Font size binding</p>

      <!-- Multiple style binding -->
      <div [style.background-color]="bgColor"
           [style.padding.px]="padding"
           [style.border-radius.px]="borderRadius">
        Styled box
      </div>
```

```
      <!-- ngStyle for multiple styles -->
      <div [ngStyle]="currentStyles">Dynamic styles</div>

      <button (click)="toggleHighlight()">Toggle Highlight</button>
    </div>
  `,
  styles: [`
    .highlight {
      background-color: yellow;
      font-weight: bold;
    }
    .box {
      border: 2px solid #2196F3;
    }
    .rounded {
      border-radius: 8px;
    }
  `]
})
export class StyleBindingComponent {
  isHighlighted: boolean = false;
  currentClasses: string = 'box rounded';

  textColor: string = '#e91e63';
  fontSize: number = 18;

  bgColor: string = '#e3f2fd';
  padding: number = 20;
  borderRadius: number = 10;

  currentStyles = {
    'background-color': '#fff3cd',
    'border': '2px solid #ffc107',
    'padding': '15px',
    'margin': '10px 0'
  };

  toggleHighlight(): void {
    this.isHighlighted = !this.isHighlighted;
  }
}
```

Output:

Browser Display:

Class and Style Binding

This text can be highlighted

Multiple classes applied

Colored text

Font size binding

Styled box

Dynamic styles

Toggle Highlight

Note: Clicking "Toggle Highlight" will add/remove yellow background from the first paragraph.

Event Binding

Event binding uses parentheses `(event)="handler()"` to listen to events like clicks, input changes, or custom events. Data flows from the view to the component.

Common Event Binding

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-event-demo',
  template: `
    <div>
      <h3>Event Binding Examples</h3>
```

```
      <!-- Click event -->
      <button (click)="handleClick()">Click Me</button>
      <p>Button clicked {{ clickCount }} times</p>

      <!-- Input event -->
      <input type="text" (input)="onInputChange($event)" placeholder="Type something...">
      <p>You typed: {{ inputValue }}</p>

      <!-- Change event -->
      <select (change)="onSelectChange($event)">
        <option value="">Select a color</option>
        <option value="red">Red</option>
        <option value="blue">Blue</option>
        <option value="green">Green</option>
      </select>
      <p>Selected color: {{ selectedColor }}</p>

      <!-- Mouse events -->
      <div (mouseenter)="onMouseEnter()"
           (mouseleave)="onMouseLeave()"
           [style.background-color]="isHovered ? '#e3f2fd' : '#fff'"
           style="padding: 20px; border: 1px solid #ddd;">
        Hover over me!
      </div>

      <!-- Keyboard event -->
      <input type="text"
             (keyup.enter)="onEnterPress($event)"
             placeholder="Press Enter...">
      <p>{{ enterMessage }}</p>
    </div>
  `
})
export class EventDemoComponent {
  clickCount: number = 0;
  inputValue: string = '';
  selectedColor: string = '';
  isHovered: boolean = false;
  enterMessage: string = '';

  handleClick(): void {
    this.clickCount++;
    console.log('Button clicked:', this.clickCount);
  }

  onInputChange(event: Event): void {
```

```
    const target = event.target as HTMLInputElement;
    this.inputValue = target.value;
  }

  onSelectChange(event: Event): void {
    const target = event.target as HTMLSelectElement;
    this.selectedColor = target.value;
    console.log('Selected:', this.selectedColor);
  }

  onMouseEnter(): void {
    this.isHovered = true;
  }

  onMouseLeave(): void {
    this.isHovered = false;
  }

  onEnterPress(event: KeyboardEvent): void {
    const target = event.target as HTMLInputElement;
    this.enterMessage = `You entered: ${target.value}`;
    target.value = '';
  }
}
```

Output:

Browser Display:

Event Binding Examples

Click Me

Button clicked 0 times

You typed:

Select a color

Selected color:

Hover over me!

Interactive behaviors:

- Click button to increment counter
- Type in first input to see real-time updates
- Select a color from dropdown
- Hover over box to change background
- Press Enter in last input to display message

Two-Way Binding

Two-way binding combines property binding and event binding to create a bidirectional data flow. It uses the banana-in-a-box syntax `[(ngModel)]` and requires the FormsModule.

Setting Up FormsModule

```
// app.module.ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';  // Import FormsModule

import { AppComponent } from './app.component';
import { TwoWayBindingComponent } from './two-way-binding.component';

@NgModule({
  declarations: [
    AppComponent,
    TwoWayBindingComponent
  ],
  imports: [
    BrowserModule,
    FormsModule  // Add FormsModule here
  ],
```

```
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Basic Two-Way Binding

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-two-way-demo',
  template: `
    <div>
      <h3>Two-Way Binding Demo</h3>

      <!-- Text input with two-way binding -->
      <label>Enter your name:</label>
      <input type="text" [(ngModel)]="userName">
      <p>Hello, {{ userName }}!</p>

      <!-- Textarea with two-way binding -->
      <label>Message:</label>
      <textarea [(ngModel)]="message" rows="4"></textarea>
      <p>Character count: {{ message.length }}</p>

      <!-- Checkbox with two-way binding -->
      <label>
        <input type="checkbox" [(ngModel)]="isAgreed">
        I agree to terms and conditions
      </label>
      <p>Agreement status: {{ isAgreed ? 'Agreed' : 'Not agreed' }}</p>

      <!-- Radio buttons with two-way binding -->
      <div>
        <label>
          <input type="radio" [(ngModel)]="selectedOption" value="option1">
          Option 1
        </label>
        <label>
          <input type="radio" [(ngModel)]="selectedOption" value="option2">
          Option 2
        </label>
        <label>
          <input type="radio" [(ngModel)]="selectedOption" value="option3">
```

```
          Option 3
        </label>
      </div>
      <p>Selected: {{ selectedOption }}</p>

      <!-- Select dropdown with two-way binding -->
      <select [(ngModel)]="selectedCountry">
        <option value="">Select country</option>
        <option value="USA">United States</option>
        <option value="UK">United Kingdom</option>
        <option value="Canada">Canada</option>
      </select>
      <p>Country: {{ selectedCountry || 'None' }}</p>
    </div>
  `
})
export class TwoWayDemoComponent {
  userName: string = '';
  message: string = '';
  isAgreed: boolean = false;
  selectedOption: string = 'option1';
  selectedCountry: string = '';
}
```

Output:

Browser Display:

Two-Way Binding Demo

Enter your name:

Hello, !

Message:

Character count: 0

I agree to terms and conditions

Agreement status: Not agreed

◉
Option 1
◯
Option 2
◯
Option 3

Selected: option1

Select country v

Country: None

Two-way binding features:

- Changes in the input update the component property immediately
- Changes in the component property update the view automatically
- Eliminates need for manual event handling for simple cases

Practical Two-Way Binding Example

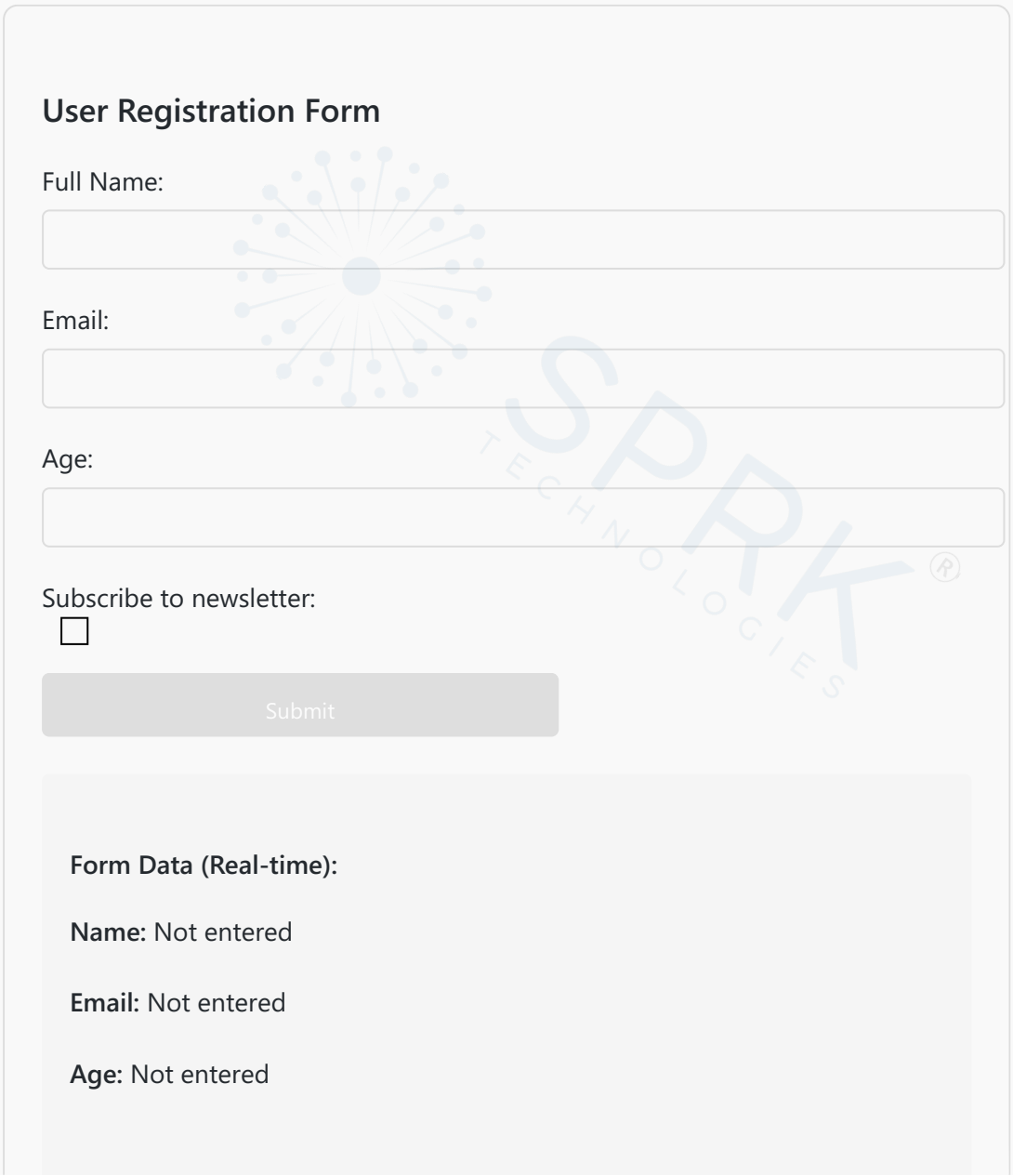
```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user-form',
  template: `
    <div style="max-width: 500px; padding: 20px; border: 1px solid #ddd; border-radius: 8px;">
      <h3>User Registration Form</h3>

      <div style="margin-bottom: 15px;">
        <label>Full Name:</label>
        <input type="text" [(ngModel)]="user.fullName" style="width: 100%; padding: 8px;
margin-top: 5px;">
      </div>

      <div style="margin-bottom: 15px;">
```

```
        <label>Email:</label>
        <input type="email" [(ngModel)]="user.email" style="width: 100%; padding: 8px; margin-
top: 5px;">
      </div>

      <div style="margin-bottom: 15px;">
        <label>Age:</label>
        <input type="number" [(ngModel)]="user.age" style="width: 100%; padding: 8px; margin-
top: 5px;">
      </div>

      <div style="margin-bottom: 15px;">
        <label>Subscribe to newsletter:</label>
        <input type="checkbox" [(ngModel)]="user.subscribe" style="margin-left: 10px;">
      </div>

      <button (click)="submitForm()"
              [disabled]="!isFormValid()"
              style="padding: 10px 20px; background-color: #4CAF50; color: white; border: none;
border-radius: 4px; cursor: pointer;">
        Submit
      </button>

      <div style="margin-top: 20px; padding: 15px; background-color: #f5f5f5; border-radius:
4px;">
        <h4>Form Data (Real-time):</h4>
        <p><strong>Name:</strong> {{ user.fullName || 'Not entered' }}</p>
        <p><strong>Email:</strong> {{ user.email || 'Not entered' }}</p>
        <p><strong>Age:</strong> {{ user.age || 'Not entered' }}</p>
        <p><strong>Newsletter:</strong> {{ user.subscribe ? 'Yes' : 'No' }}</p>
      </div>
    </div>
  `
})
export class UserFormComponent {
  user = {
    fullName: '',
    email: '',
    age: null,
    subscribe: false
  };

  isFormValid(): boolean {
    return !!(this.user.fullName && this.user.email && this.user.age);
  }

  submitForm(): void {
```

```
    console.log('Form submitted:', this.user);
    alert(`Registration successful!\nName: ${this.user.fullName}\nEmail: ${this.user.email}`);
  }
}
```

Output:

Browser Display:



Newsletter: No

Features demonstrated:

- All form fields use two-way binding with [(ngModel)]
- Real-time preview shows data as you type
- Submit button is disabled until all required fields are filled
- Form data is displayed and updated automatically

Binding to Component Properties and Methods

Data binding can target not only DOM elements but also component properties and methods, enabling communication between parent and child components.

Parent-Child Component Binding

```
// child.component.ts
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `
    <div style="border: 2px solid #2196F3; padding: 15px; margin: 10px 0;">
      <h4>Child Component</h4>
      <p>Received from parent: {{ dataFromParent }}</p>
      <button (click)="sendToParent()">Send Data to Parent</button>
    </div>
  `
})
export class ChildComponent {
  @Input() dataFromParent: string = '';
  @Output() dataToParent = new EventEmitter<string>();

  sendToParent(): void {
    this.dataToParent.emit('Hello from child component!');
  }
}
```

```
// parent.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-parent',
  template: `
    <div>
      <h3>Parent Component</h3>

      <!-- Property binding to child Input -->
      <app-child
        [dataFromParent]="parentMessage"
        (dataToParent)="receiveFromChild($event)">
      </app-child>

      <p>Message from child: {{ childMessage }}</p>

      <input type="text" [(ngModel)]="parentMessage"
             placeholder="Type to send to child..."
             style="width: 300px; padding: 8px;">
    </div>
  `
})
export class ParentComponent {
  parentMessage: string = 'Hello from parent!';
  childMessage: string = '';

  receiveFromChild(message: string): void {
    this.childMessage = message;
    console.log('Received from child:', message);
  }
}
```

Output:

Browser Display:

Parent Component

Child Component

Received from parent: Hello from parent!

Send Data to Parent

Message from child:

Hello from parent!

Component communication:

- Parent sends data to child via property binding [@Input]
- Child sends data to parent via event binding [@Output]
- Typing in input updates child component in real-time
- Clicking button in child sends message to parent

Data Binding Comparison

Binding Type	Syntax	Data Flow	Use Case
String Interpolation	`{{ value }}`	Component → View	Display text content
Property Binding	`[property]="value"`	Component → View	Set element properties
Event Binding	`(event)="handler()"`	View → Component	Handle user actions
Two-way Binding	`[(ngModel)]="value"`	Component ↔ View	Form inputs, editable data

Best Practices

- **Use interpolation** for simple text display
- **Use property binding** when setting element properties or attributes
- **Use event binding** to respond to user actions
- **Use two-way binding** sparingly - only for form controls where bidirectional sync is needed
- **Import FormsModule** when using [(ngModel)]
- **Keep binding expressions simple** - move complex logic to component methods
- **Use @Input and @Output** for parent-child component communication

Summary

Angular's data binding mechanisms provide powerful ways to connect components and views:

- **String Interpolation** displays component data using `{{ }}`
- **Property Binding** sets element properties dynamically with `[property]`
- **Event Binding** responds to user interactions with `(event)`
- **Two-way Binding** synchronizes data bidirectionally with `[(ngModel)]`
- **Component Binding** enables parent-child communication via @Input and @Output

Mastering these binding techniques is essential for building interactive, data-driven Angular applications.